

What Is Docker?

Run applications with a defined runtime and configuration.

Learning Goals

- Explain the problem Docker solves: one application, many different environments.
- Distinguish a Dockerfile, an image, and a container.
- Run a command in a temporary container with `docker run`.

The Problem

To run PHP or Laravel, we must install the required runtime, tools, and dependencies on a server.

- These installations can change the server's environment and cause conflicts with other applications.
- A project may require a specific version, such as PHP 8.4, which may differ from the version already installed.

The Solution - Docker

Docker packages an application and its dependencies into an isolated container.

- This keeps the server environment clean and helps the application run consistently across different computers.
- Docker allows us to install any applications with any versions without conflict with other tools.

Laravel on Docker

A Laravel app typically depends on PHP, PHP extensions, a database such as MySQL, and application configuration.

- Different installations can produce different results.
- Dockerfiles and Compose files describe these requirements in a reusable form.
- You still need correct settings, working code, and compatible images.

Module 2 and Module 3

In Module 2, we installed Laravel and a SQL database on a local computer and deployed the application.

- An application may work on one computer but fail on another because their environments differ.
- Docker defines the application's environment, helping it run consistently across systems.
- In Module 3, we will run a Laravel API and MySQL with Docker.

Three Terms

Term	Meaning
Dockerfile	Instructions for building an image
Image	Packaged runtime and application files
Container	A running instance of an image

Dockerfile → (build) → Image → (run) → Container.

It's similar to Java programming:

- source : Dockerfile
- source built into the bytecode: Image
- run on JVM as object: container

Docker uses a Dockerfile to build an image, then runs a container from that image.

Start with One Container

In the beginning, we start with a single container.

Start Docker without a Dockerfile

Start Docker Desktop, then run this command in the Terminal:

```
docker run --rm php:8.4-cli php --version
```

This command checks the PHP version in a temporary container.

- If needed, Docker downloads the `php:8.4-cli` image.
- The `php --version` command runs in a container created from the image.
- The container is removed when the command ends; the image remains.

The message from the Docker shows the process.

```
Unable to find image 'php:8.4-cli' locally
8.4-cli: Pulling from library/php
cb737533f4f7: Pull complete
3c3cce82d972: Pull complete
6ddf914d0681: Pull complete
f367ad55561b: Pull complete
77827b267789: Pull complete
07a0633b2fc2: Download complete
493e6bd218a3: Download complete
Digest: sha256:0177c7589c0715b63e5343342a7c1f1887db759ad0b0d6ad3bb949938bba6b47
Status: Downloaded newer image for php:8.4-cli
PHP 8.4.26 (cli) (built: Sep 24 2026 19:11:07) (NTS)
Copyright (c) The PHP Group
Built by https://github.com/docker-library/php
Zend Engine v4.4.26, Copyright (c) Zend Technologies
```

Use Docker Desktop to check that **PHP8.4** image is installed on your local computer.

Isolated PHP8.4 without Installation

If the PHP 8.4 image is not already local, Docker downloads it from a registry and stores it on your computer.

- Docker runs the image in a container.
- The container is isolated from the host, but mounted files can still be changed.
- This lets us run PHP in an isolated container without installing it on the host.